MPI-ML: A High-Performance Sparse Communication Layer for Machine Learning

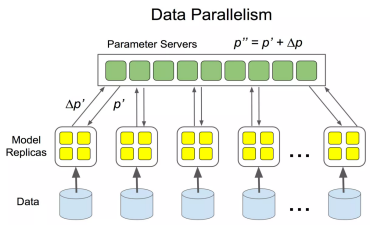
Cedric Renggli — ETH Zurich
Dan Alistarh — IST Austria
Torsten Hoefler — ETH Zurich

Communication in Machine Learning Workloads

Data-parallel Machine Learning

- ***Data*** **shared among multiple compute nodes**
- **The *set of parameters (model)* is consistent across machines**
- **Communication and Synchronization are often bottlenecks**

Example:



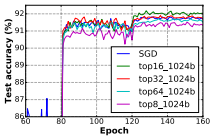
- **Challenge**: Reduce communication and synchronization overheads
- **Idea**: Exploit **robustness to asynchrony** and **quantization noise**

Many settings have naturally-sparse communication!

- *Linear regression via Stochastic Gradient Descent*:
  - *Stochastic gradient:*

$$\tilde{g}(x_t) = a_t(a_t^T x_t - b_t)$$

  - If samples a_t are *sparse*, the update is *sparse*!
  - Occurs in many real-world datasets.
- *Top-k SGD* [Dryden et al., Aji & Heafield]
  - Idea: only communicate some top percentage of the gradient
  - Save the rest locally
  - Accuracy on CIFAR-10:

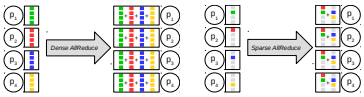


Leveraging sparsity is a challenge, since there is no MPI support!

MPI-ML: An Overview

Main Idea

- **Efficient support for sparse communication**
- **MPI-like operation semantics on top of sparse streams**
- **Allow non-blocking operations and quantized communication**



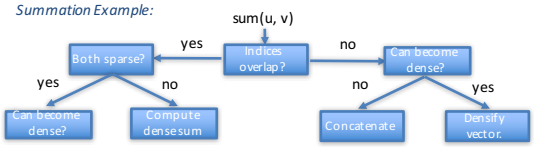
(a) Dense AllReduce (b) Sparse AllReduce

- **We support sparse AllReduce, Reduce, and AllGather operations**

Sparse Streams

The basic data representation behind MPI-ML.

- **Starts from a standard index-based sparse representation.**
- *Summation Example:*

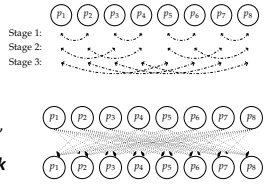


Key: fast switch from *sparse* to *dense* vectors.

Efficient Sparse Collectives: the Static Case

Setting: in this case, the size of the resulting data remains below the threshold δ, where the sparse representation is still efficient.

Case 1: Latency-Dominated.
We use **recursive doubling**:
In the ith round, nodes that are distance 2^{i-1} apart exchange all data.
Runtime: $\alpha \log P + \beta (P-1) k$



Case 2: Bandwidth-Dominated.
We **split** the dimension into P partitions, and then perform a **sparse AllGather:**
Runtime: $\alpha \log P + \alpha (P-1) + \beta P k$

Efficient Sparse Collectives: General Solution

- The Dynamic Case
  - The result becomes **dense** during addition
  - We use the solution from the **bandwidth-dominated case**
  - Communicate data in **sparse format**, then broadcast **dense**

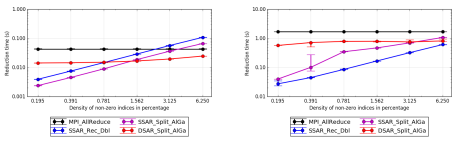
- Quantized Communication
  - We implement **QSGD quantization** [Alistarh et al.]
  - Allows sent values to be rounded to arbitrary precision
  - Provably converges, and provides additional compression

- Non-blocking collectives
  - **Threads can proceed** even if the collective has not completed
  - Allows us to overlap **communication** and **computation**

Performance

Target Architectures:

- **Supercomputing**: CSCS Piz Daint (3rd worldwide, 1st in Europe)
- **Cloud computing**: Amazon EC2 and research cluster



(a) Piz Daint (b) Greina (GigE)

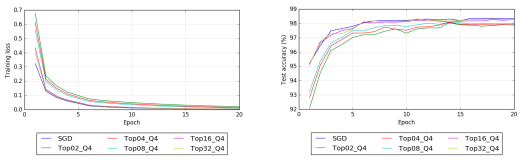
Figure 4: Data density versus reduction time for various algorithms, dimension $N = 16$M and $P = 8$ nodes.

System	Dataset	Model	# of nodes	Baseline Time (s)	Algorithm	Algo. Time (s)	Speedup
Piz Daint	Webspam	LR LR SVM	32 128 32	2.40 (2.16) 0.64 (0.58) 1.62 (1.42)	SSAR_Recursive_Double	0.68 (0.35) 0.18 (0.10) 0.65 (0.44)	**3.53 (6.17)** **3.56 (5.80)** **2.49 (3.23)**
Piz Daint	URL	LR LR SVM	32 128 32	2.64 (2.58) 0.57 (0.56) 1.98 (1.93)	SSAR_Recursive_Double	0.75 (0.70) 0.29 (0.28) 0.56 (0.53)	**3.52 (3.69)** **1.97 (2.00)** **3.54 (3.64)**
Piz Daint	Webspam URL	LR LR	8 8	4.67 (3.79) 3.77 (3.53)	SSAR_Split_AllGather	2.56 (1.58) 2.09 (1.50)	**1.82 (2.40)** **1.80 (2.35)**
Greina (IB)	Webspam URL	LR LR	8 8	6.52 (4.67) 8.14 (4.47)	SSAR_Split_AllGather	3.63 (1.90) 6.11 (2.49)	**1.80 (2.46)** **1.33 (1.80)**
Greina (GigE)	Webspam URL	LR LR	8 8	76.80 (75.95) 104.50 (100.46)	SSAR_Split_AllGather	3.79 (2.95) 8.26 (4.22)	**20.26 (25.75)** **12.65 (23.81)**

Table 2: Distributed optimization using MPI-OPT. The times represent average time in seconds for a full dataset pass, and its communication part in brackets. Speedup versus dense MPI is shown end-to-end, with communication speedup in brackets.

Target Applications:

- Large-scale **distributed optimization** (SGD and SCD for regression)
- Training **large neural networks** (CNNs and RNNs)



(a) Training error (b) Test accuracy

Figure 5: MNIST convergence for various algorithm variants.

System	Dataset	Model	# of nodes	Baseline Time (m)	Algorithm	Algo. Time (m)	Speedup
Piz Daint	ImageNet	VGG19	8	61.23 (32.72)	Q4	39.49 (9.87)	**1.55 (3.31)**
Piz Daint	ImageNet	AlexNet	16	33.69 (26.82)	Q4	25.83 (19.77)	**1.30 (1.36)**
Piz Daint EC2	MNIST	MLP	8	1.78 (1.65) 24.41 (24.18)	Top16_Q4 Top16_Q4	0.49 (0.36) 1.28 (1.05)	**3.65 (4.53)** **19.12 (22.97)**

Table 3: Training neural networks using CNTK. The times represent average time in minutes for a full dataset pass, and its communication part in brackets. Speedup versus dense MPI is shown end-to-end, with communication speedup in brackets.

Contact: dan.alistarh@ist.ac.at

ETH zürich

IST AUSTRIA
Institute of Science and Technology